Data Science Capstone Project Report: SpaceX Falcon 9 Landing Prediction

1. Executive Summary & Business Objectives

1.1 Project Background

The commercial aerospace industry has experienced a paradigm shift driven by SpaceX's disruptive pricing model. By successfully recovering and reusing the Falcon 9 rocket's first stage, SpaceX offers launch services at an advertised cost of approximately $62 million. This drastically undercuts traditional aerospace providers whose single-use rocket infrastructure costs often exceed $165 million. The core of SpaceX's economic dominance relies entirely on the operational reliability and success rate of its booster recoveries.

1.2 Business Problem & Objectives

For any competing aerospace startup, predicting the operational success of SpaceX's rocket landings is essential to accurately reverse-engineer their launch cost structures and financial margins.
The objective of this applied data science project is to build a robust predictive machine learning pipeline that determines whether a Falcon 9 first stage will successfully land based on telemetry, orbital, and historical parameters. By achieving a high-accuracy binary classification model, a competitor can simulate SpaceX's cost-efficiency per mission, optimize its own financial bidding strategy, and make data-driven commercial decisions.

2. Data Collection & Preparation

2.1 API Data Acquisition

For data acquisition, we used the Python requests library to send GET requests to the official SpaceX REST API. Raw flight data was received in JSON format. To structure this information for our study, we converted this JSON stream into a Pandas DataFrame. This transformation was essential because the DataFrame provides a flexible environment for data manipulation and cleaning.
During this phase, we applied a critical filter to isolate only Falcon 9 rocket launches. Furthermore, since many complex variables (such as launch sites or payload characteristics) were initially provided as identifiers (numeric IDs), we had to perform secondary API requests to unpack these nested structures into usable textual data.

2.2 Web Scraping from Wikipedia

To enrich and cross-reference our dataset, we performed web scraping on the historical Falcon 9 launch records available on Wikipedia. We utilized the `BeautifulSoup` library to parse the raw HTML code into a navigable Document Object Model (DOM) tree. Our programmatic logic implemented nested loops to isolate structural HTML elements. Specifically, we used the `<table>` tag to locate relevant launch tables, `<tr>` tags to iterate through individual rows, and `<td>` tags to extract data cells representing features like flight numbers, dates, and payload details. During extraction, custom string-filtering functions were applied to clean the text, systematically ignoring extra whitespaces and stripping out Wikipedia citation brackets (e.g., `[1]`) to isolate pure, standardized string records.

2.3 Data Wrangling & Target Engineering

To prepare the dataset for predictive modeling, several critical data wrangling procedures were executed. First, we engineered the primary target variable, `Class`, by mapping complex categorical landing outcomes (e.g., `True Ocean`, `False RTLS`, `None`) into a standardized binary classification format, where `1` represents a successful landing and `0` represents a failure.
Second, to address missing data without discarding valuable flight records, we applied mean imputation to the `PayloadMass` column. Any `NaN` values were replaced with the calculated average payload mass of the dataset.
Finally, because machine learning algorithms require strictly numerical inputs, we performed One-Hot Encoding on categorical features such as `Orbit`, `LaunchSite`, and `BoosterVersion`. This technique transformed each distinct category into its own binary column (containing `1` for True or `0` for False), ensuring the predictive models could correctly interpret non-numerical relationships.

3. Exploratory Data Analysis & Geospatial Insights

3.1 SQL Data Exploration

To simulate an enterprise-level data pipeline, the cleaned dataset was loaded into an IBM Db2 relational database. While Pandas is excellent for in-memory data manipulation, SQL (Structured Query Language) is the industry standard for querying large-scale databases directly at the server level, providing efficient data extraction.
We utilized SQL queries to isolate critical launch trends and metrics before passing them to our visualization tools. Key operational insights extracted via SQL included:

- Customer Analysis: Calculating the total payload mass carried for specific clients, such as NASA, utilizing aggregation functions like `SUM(PayloadMass)`.
- Historical Milestones: Identifying the exact date of the first successful ground pad landing by applying the `MIN(Date)` function combined with specific `WHERE` filtering clauses.
- Advanced Filtering: Leveraging subqueries to isolate specific flight profiles, launch sites, and orbit success rates.

3.2 Geospatial Analysis & Logistics

To understand the geographic strategy behind SpaceX's operations, we utilized the `folium` library to generate interactive maps of all active launch facilities. Spatial visualization immediately highlighted two critical operational constraints for site selection: safety (strict proximity to the coastline for over-water trajectories and debris mitigation) and logistics (immediate access to highways and railways for transporting heavy rocket boosters).
To manage visual data clutter caused by mapping 90 historical flights over only a few launch pads, we implemented the `MarkerCluster` plugin. This feature

dynamically grouped overlapping coordinates, ensuring a responsive and readable dashboard. Furthermore, we utilized the `PolyLine` function combined with the Haversine formula to map and calculate the exact distances between launch pads and critical infrastructure, mathematically proving the strategic advantage of these geographic locations.

4. Predictive Modeling & Evaluation

4.1 Data Preprocessing & Leakage Prevention

To rigorously evaluate our machine learning models, the dataset was partitioned into training and testing sets utilizing the `train_test_split` function, with a dedicated sample of 18 records reserved strictly for out-of-sample testing. A critical methodological step was applying feature standardization (via `StandardScaler`) after the split. By fitting the scaler exclusively on the training data and subsequently transforming the test data, we successfully prevented data leakage. This ensured the test set remained completely unseen, preserving the statistical integrity of our evaluation.

4.2 The Overfitting Phenomenon (Decision Tree)

During our modeling phase, we trained a Decision Tree classifier. While it achieved an impressive baseline cross-validation accuracy of 85.89% on the training set, its performance severely degraded to 77.78% when exposed to the test data. This significant variance is a classic manifestation of overfitting. The algorithm essentially memorized the training noise rather than learning generalized, underlying patterns, making it too brittle for real-world production.

4.3 Model Selection & Business Risk Assessment

Despite Logistic Regression, SVM, and KNN achieving an identical test accuracy of 83.33%, Logistic Regression was definitively selected as the production model due to its high interpretability. Unlike the "black-box" nature of SVM or the purely distance-based mechanics of KNN, Logistic Regression provides extractable feature weights and exact outcome probabilities. This transparency allows telemetry engineers to audit which physical variables impact the landing risk and enables the financial team to integrate these exact probabilities into dynamic pricing algorithms.

Confusion Matrix & Financial Cost of Errors:

An analysis of the final Logistic Regression confusion matrix reveals strong recall for successes (12 True Positives, 0 False Negatives) but highlights 3 False Positives (predicting a successful landing when the actual outcome was a failure). In the context of aerospace economics, these False Positives represent a significant financial risk. Predicting a successful recovery might cause "Space Y" to underbid a contract; if that booster subsequently crashes, the company incurs an unexpected replacement cost of tens of millions of dollars. Future iterations of this model must prioritize lowering this False Positive rate to protect profit margins.